

Estructura de Datos y Algoritmos.
Práctica 2021-2022. Fase 2.
EHU-UPV

Judith Antelo Lacalle, Silvia Arenales Muñoz y Xabier Badiola Guembe

8 de diciembre de 2021

Índice general

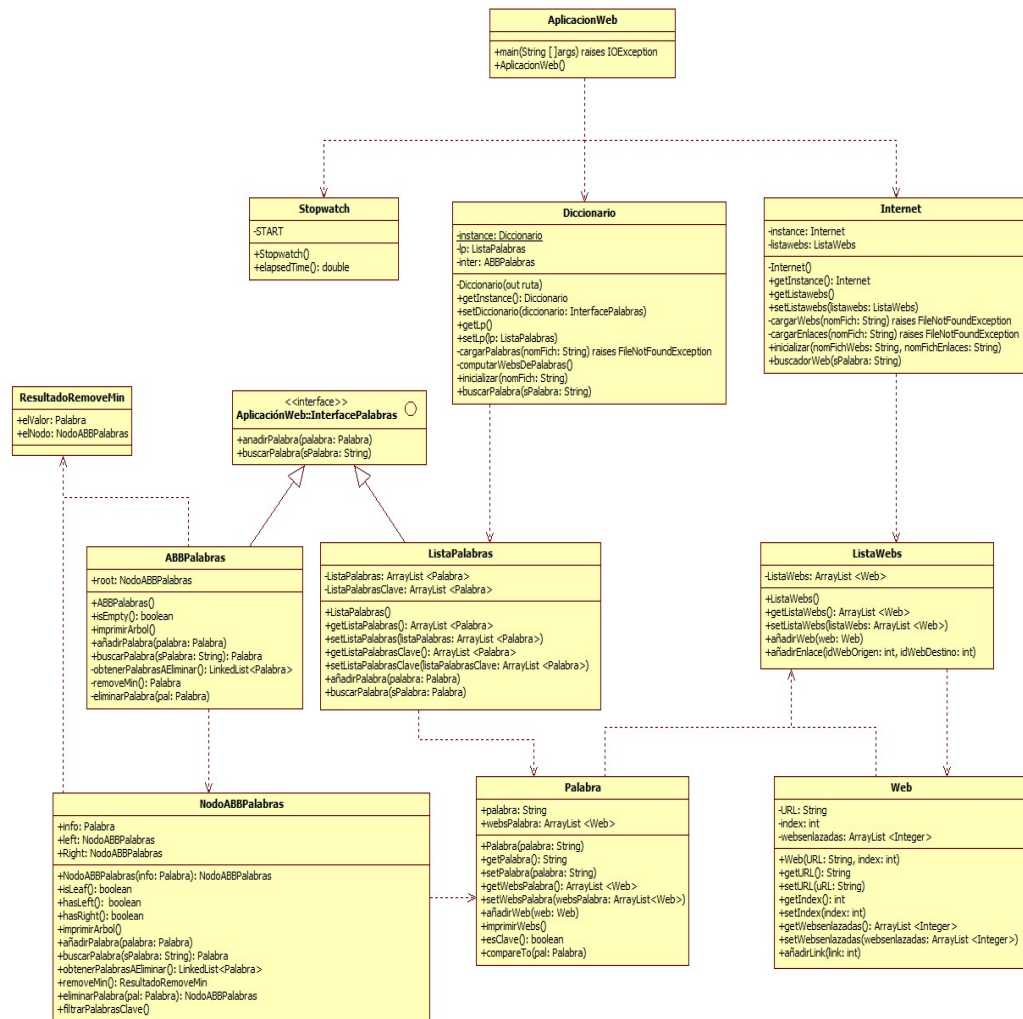
1. Diagrama de clases.	1
1.1. UML	1
2. Métodos implementados.	2
2.1. Interfaz InterfacePalabras	3
2.2. Clase Diccionario	4
2.3. Clase Palabra	9
2.4. Clase ABBPalabras y Clase NodoABBPalabras	10
3. Respuesta al apartado f.	20

Capítulo 1

Diagrama de clases.

1.1. UML

A continuación se muestra un esbozo del diagrama que representa nuestro escenario:



Capítulo 2

Métodos implementados.

A continuación se explicarán cada método desarrollado para la realización de la fase 2 del proyecto y a su vez, se determinara el orden temporal de cada uno de ellos.

2.1. Interfaz InterfacePalabras

En esta 2ª fase del proyecto hacemos uso de un interfaz, InterfacePalabras.

- Algoritmo:

```
public interface InterfacePalabras {  
  
    public void anadirPalabra(Palabra palabra);  
    public Palabra buscarPalabra(String sPalabra);  
}
```

Listing 2.1: Interfaz implementada por las distintas representaciones.

Esta, como vemos en el algoritmo, tiene especificado solo dos métodos abstractos.

Más tarde las clases ListaPalabras y ABBPalabras implementan esta interfaz por lo tanto deberán proporcionar una implementación para estos dos métodos declarados en la interfaz.

2.2. Clase Diccionario

La clase Diccionario representa un listado de palabras clave. En esta fase, las cargamos de el fichero *wordsshuffle.txt* el cual está desordenado y además es de mayor longitud en comparación con la anterior fase.

Primero de todo en esta clase, hemos instanciado un objeto de tipo de la Interfaz creada anteriormente.

```
private InterfacePalabras inter;
```

Listing 2.2: Declaración interfaz.

▪ setDiccionario

- Algoritmo:

```
public void setDiccionario(InterfacePalabras diccionario) {  
    this.inter = diccionario;  
}
```

Listing 2.3: Inicializa el diccionario con el diccionario que se le pasa como parámetro.

Hemos creado un nuevo método para inicializar el diccionario, ya que tenemos un nuevo atributo de tipo interfaz.

▪ cargarPalabras

- Algoritmo:

```
/**  
 * Carga el diccionario desde el fichero indicado  
 * @param nomFich: nombre del fichero que contiene el  
 * diccionario  
 */  
  
private void cargarPalabras(String nomFich) {  
  
    try {  
        FileReader myObj = new FileReader(nomFich);  
        Scanner myReader = new Scanner(myObj);  
  
        while (myReader.hasNextLine()) {  
            String pal = myReader.nextLine();  
            inter.anadirPalabra(new Palabra(pal));  
        }  
        myReader.close();  
    } catch (FileNotFoundException e) {  
        System.out.println("Ha habido un error.");  
        e.printStackTrace();  
    }  
}
```

Listing 2.4: Carga las palabras del fichero.

Se lee el fichero de entrada, que será el fichero *wordsshuffle.txt*, y se cargarán todas las palabras por medio del método *anadirPalabra*.

Pero en este caso, este método llamada al método *anadirPalabra* mediante la instancia de la interfaz, es decir está llamando a las clases que contienen este método implementada por la interfaz. Es decir, está llamando al método *anadirPalabra* de la *ListaPalabras* y al de *ABBPalabra*.

En la clase *ListaPalabras* añadirá a un array mientras que en la clase *ABBPalabra* lo añadirá a un árbol.

- Casos de prueba considerados:

Entrada	Función
Ficheros <i>wordsshuffle.txt</i>	Carga las palabras del contenido en el fichero indicado.
Vacío	Se elevará la excepción.

- Orden temporal:

El orden temporal es $O(N)$. Carga las webs contenidos en el fichero *nomFich* por tanto se recorrerá todo el fichero. Por tanto es de orden N , siendo N la longitud del fichero.

■ computarWebsDePalabras

- Algoritmo:

```
/**
 * Asigna a cada palabra del diccionario las webs a las que
 * hace referencia Pre:
 * Internet y el diccionario ya est n cargados
 */

private void computarWebsDePalabras() {

    ListaWebs lw = Internet.getInstance().getListawebs();

    for (Web w : lw.getListawebs()) {

        String[] arrayi = w.getURL().split("\\.");

        if (arrayi[1].length() > 3) {
            // descomponemos el string del URL en substring y lo
            // metemos en una lista
            for (int a = 3; a < 11; a++) {

                for (int i = 0; a <= arrayi[1].length() - i && a <
20; i++) {

                    Palabra p = inter.buscarPalabra(arrayi[1].substring
(i, a + i));

                    if (p != null) {
                        p.anadirWeb(w);
                    }
                }
            }
        }
    }
}
```

Listing 2.5: Asigna a cada palabra del diccionario las webs a las que hace referencia.

Al ser el listado de las webs el listado más largo, nos interesa recorrerlo una sola vez. Para ello, tenemos que verificar que en la URL de la web, es decir en su atributo string, haber si tiene alguna palabra clave del diccionario.

Para ello primero, dividimos el String en función de que nos quedemos solamente con la parte de la mitad de la URL, ya que la primera parte (*www.*) y la última parte (*.com/.es...*) no nos interesa a la hora de buscar una palabra clave. Esto lo realizamos mediante el *.split()*.

Vamos analizando primeramente en substrings de 3 en 3 hasta el fin del string verificando si existe alguna palabra clave, luego aumentamos de 4 en 3 hasta que sea de 10 en 10, ya que es la longitud de una palabra clave.

Para verificar si ese substring es una palabra del diccionario, llamamos al método buscarPalabra.

Pero como en el caso anterior, este método llamada al método buscarPalabra mediante la instancia de la interfaz, es decir está llamando a las clases que contienen este método implementada por la interfaz. Es decir, está llamando al método buscarPalabra de la ListaPalabras y al de ABBPalabra.

En la clase ListaPalabras buscará de forma dicotómica la palabra en la lista la cual está desordenada pues no es nada efectiva, mientras que en la clase ABBPalabras buscará de forma dicotómica pero en un árbol binario, pero en este caso sí es efectivo ya que se ha añadido las palabras de forma ordenada.

- Casos de prueba considerados:

Entrada	Función
Si las listas de webs o palabras	Si están vacías no se computarán ninguna web.
En caso contrario	Se computarán las webs que la lógica del programa considere.

- Orden temporal:

El orden temporal es $O(\log M * N)$. En este método recurrimos al método buscarPalabras el cual es de orden $\log M$. Además iteramos N veces para recorrer las webs y luego $(7 * x - 42)$ para recorrer el string de las URLs por medio de los substrings, pero este podemos considerarlo de orden 1 ya pues por el contraste de tamaños de ciclos recorridos.

■ buscarPalabra

- Algoritmo:

```
/**
 * Busca una palabra en el diccionario y la devuelve
 * @param sPalabra: texto de la palabra a buscar
 * @return la Palabra (si est  en el diccionario), null en
 * caso contrario
 */
public Palabra buscarPalabra(String sPalabra) {
    return this.inter.buscarPalabra(sPalabra);
}
```

Listing 2.6: Busca el String de una palabra en el diccionario y devuelve la palabra.

La diferencia que tiene con la fase anterior es que llamamos a la interfaz, y así hace uso también de método buscarPalabra de la clase ABBPalabras.

- Casos de prueba considerados:

Entrada	Función	Devuelve
adult	La busca en el diccionario y está. sus direcciones asociadas.	Devuelve la palabra con
asdf	No está en la lista.	null.

- Orden temporal:

El orden temporal es $O(\log N)$. Como hemos explicado previamente, que era una búsqueda dicotómica, es ese su orden temporal ya que hace uso del método buscarPalabra(string).

2.3. Clase Palabra

La clase palabra representa las palabras del diccionario, que pueden ser de tipo clave o no y cada palabra tiene un array, el cual almacena las diferentes URLs a las que está conectada.

- **compareTo**

- Algoritmo:

```
@Override
public int compareTo(Palabra pal) {
    int l1 = pal.palabra.length();
    int l2 = this.palabra.length();
    int lmin = Math.min(l1, l2);

    for (int i = 0; i < lmin; i++) {
        int str1 = (int)pal.palabra.toLowerCase().charAt(i);
        int str2 = (int)this.palabra.toLowerCase().charAt(i);

        if (str1 != str2) {
            return str1 - str2;
        }
    }

    if (l1 > l2) return 1;
    else if (l1 < l2) return -1;
    else return 0;
}
```

Listing 2.7: Compara dos Palabras en función de sus Strings.

En esta clase hemos tenido que sobrescribir el método compareTo, para que pudiese comparar dos Palabras en función de su atributo String. Así de este modo, podremos crear un árbol binario, es decir un árbol ordenado. Luego su acceso es más rápido.

- Casos de prueba considerados:

Entrada	Función	Devuelve
adult y from	adult al empezar con A estará antes que la F de from.	-1.
from y adult	adult al empezar con A estará después que la F de from.	1.

- Orden temporal:

El orden temporal sería de $O(N \cdot M)$ con N = número de letras en uno de los strings y M la longitud máxima del otro String.

2.4. Clase ABBPalabras y Clase NodoABBPalabras

El ABBPalabras, es el árbol binario de búsqueda que guarda la referencia de la raíz(*root*) que será de tipo *NodoABBPalabras*. Entonces, en esta clase, en la del árbol, implementaremos al método principal(no recursivo). Mediante inmersión, llamaremos al método recursivo que está en la clase *NodoABBPalabras*. En la llamada inicial(principal), se aplicará el método recursivo al nodo raíz (*this.root*).

■ anadirPalabra

- Algoritmo **ABBPalabras**:

```
/**
 * Anade una palabra al rbol
 * @param palabra: palabra a a adir
 */

public void anadirPalabra(Palabra palabra) {
    if (this.isEmpty()) {
        this.root = new NodoABBPalabras(palabra);
    } else {
        this.root.anadirPalabra(palabra);
    }
}
```

Listing 2.8: Clase Árbol Binario.

- Algoritmo **NodoABBPalabra**:

```
public void anadirPalabra(Palabra palabra) {

    if (palabra.compareTo(this.info) < 0) { // elem debe ir
antes que el actual
        if (this.hasLeft())
            this.left.anadirPalabra(palabra);
        else
            this.left = new NodoABBPalabras(palabra);
    } else { // elem debe ir despu s que el actual
        if (this.hasRight())
            this.right.anadirPalabra(palabra);
        else
            this.right = new NodoABBPalabras(palabra);
    }
}
```

Listing 2.9: Clase Nodo .

Damos por hecho que el elemento no está en el árbol. Insertaremos en un nodo en el que al menos unos de los punteros izq o der tenga valor null. Para realizar la condición anterior se desciende en el árbol a partir del nodo raíz, dirigiéndose de izq a der de un nodo, según que el valor a insertar sea inferiro o superior al valor del campo clave *info* de este nodo. Cuando se alcanza un nodo del árbol en que no puede continuar, el nuevo elemento se engancha a la izquierda

o derecha de este nodo en función si su valor es mayor o menor (haciendo uso del `compareTo` declarado anteriormente).

- Casos de prueba considerados:

Entrada (web, URL - index) string - int	Función
animalhumanesociety.org 112153	Añade a la Lista de webs la web de entrada.
tfcvolleyball.com 1774550	Añade a la Lista de webs la web de entrada

- Orden temporal:
El coste sería de $O(1)$ si se añade en la raíz, si no sería de $O(N)$, siendo N el número de elementos.

■ buscarPalabra

- Algoritmo ABBPalabras:

```
/**
 * Busca una palabra en el rbol y la devuelve
 * @param sPalabra: texto de la palabra a buscar
 * @return la Palabra (si est en el rbol ), null en caso
 * contrario
 */
public Palabra buscarPalabra(String sPalabra) {
    if (this.isEmpty())
        return null;
    return this.root.buscarPalabra(sPalabra);
}
```

Listing 2.10: Clase Árbol Binario.

- Algoritmo NodoABBPalabras:

```
public Palabra buscarPalabra(String sPalabra) {
    Palabra res = this.info;
    Palabra p= new Palabra(sPalabra);

    if (res.compareTo(p)==0)
        return res;
    else if(p.compareTo(res)<0) {
        if (this.hasLeft())
            return this.left.buscarPalabra(sPalabra);
        else return null;
    }
    else {
        if (this.hasRight())
            return this.right.buscarPalabra(sPalabra);
        else return null;
    }
}
```

Listing 2.11: Clase Nodo.

El algoritmo de búsqueda del elemento se realiza comparándolo (compareTo) con la clave de la raíz del árbol. Si no es el mismo, se pasa al árbol izquierdo o derecho, según el resultado de la comparación, y se repite la búsqueda en ese subárbol.

- Casos de prueba considerados:

Entrada (String)	Función
adult	Busca en el árbol y la devuelve la palabra con su correspondiente array con las listas webs enlazadas.
asdf	Devuelve null, ya que no está en el árbol.

- Orden temporal:

La búsqueda en un árbol binario ordenado es dicotómica, que a cada examen de un nodo se elimina aquel de los subárboles que no contiene el valor buscado.

Por tanto, en el caso peor el orden sería de $O(N)$ y si el árbol estuviese Equilibrado $O(\log N)$, siendo N el número de elementos.

■ obtenerPalabrasAEliminar

- Algoritmo ABBPalabras:

```
/**
 * Devuelve una lista con todas aquellas palabras del arbol
 * que no sean palabra
 * clave de ninguna web
 * @return lista con las palabras a eliminar
 */

private LinkedList<Palabra> obtenerPalabrasAEliminar() {
    LinkedList<Palabra> result = new LinkedList<Palabra>();
    if (this.isEmpty())
        return result;
    return this.root.obtenerPalabrasAEliminar();
}
```

Listing 2.12: Clase Árbol Binario

- Algoritmo NodoABBPalabras:

```
public LinkedList<Palabra> obtenerPalabrasAEliminar() {
    LinkedList<Palabra> result = new LinkedList<Palabra>();

    if(!this.info.esClave())
        result.add(this.info);

    if (this.hasLeft())
        result.addAll(this.left.obtenerPalabrasAEliminar());

    if (this.hasRight())
        result.addAll(this.right.obtenerPalabrasAEliminar());

    return result;
}
```

Listing 2.13: Clase Nodo

Devuelve una lista con todas aquellas palabras del árbol que no sean palabra clave de ninguna web.

- Casos de prueba considerados:

Antes	Salida Tras la llamada al método
[dormir,aa, ane,adult, from, asdfghkl]	[aa,ane,asdfghjkl]

- Orden temporal:

La búsqueda de todas aquellas palabras que no sean palabras clave de un árbol binario de búsqueda es dicotómica.

El orden temporal es $O(N)$, siendo N el número de elementos.

■ removeMin

Para esta clase necesitamos crear una clase para devolver dos elementos: la Palabra (mínimo) y el Nodo.

```
public class ResultadoRemoveMin {
    Palabra elValor;
    NodoABBPalabras elNodo;
}
```

Listing 2.14: Guarda el valor mínimo y el apuntador al nodo.

● Algoritmo **ABBPalabras**:

```
private Palabra removeMin() {
    if(this.isEmpty()) return null;
    else {
        ResultadoRemoveMin resul=root.removeMin();
        root=resul.elNodo;
        return resul.elValor;
    }
}
```

Listing 2.15: Elimina el mínimo del árbol.

● Algoritmo **NodoABBPalabras**:

```
public ResultadoRemoveMin removeMin() {
    ResultadoRemoveMin resul = new ResultadoRemoveMin();

    if(!this.hasLeft()) { //b)
        resul.elValor= this.info;
        resul.elNodo=this.right;
    }
    else { //a)
        ResultadoRemoveMin resultadoLeft = this.left.removeMin();
        this.left=resultadoLeft.elNodo;
        resul.elValor=resultadoLeft.elValor;
        resul.elNodo=this;
    }
    return resul;
}
```

Listing 2.16: Elimina el mínimo del árbol.

a) El actual (this) tiene subárbol izquierdo, entonces el mínimo estará en el subárbol izquierdo. Hacemos una llamada recursiva para eliminar el método del subárbol izquierdo (**this.left.removeMin()**). Este devolverá dos elementos aquellos de la clase **ResultadoRemoveMin**, tanto el mínimo como el apuntador. Además, tiene que devolver el left/right del padre this.

b) actual (this) no tiene subárbol izquierdo, por tanto es el mínimo.

- Casos de prueba considerados:

Caso	Antes	Tras la llamada al método
Caso a)		
Caso b)		

- Orden temporal:
El orden temporal es $O(\log N)$ siendo N el número de elementos en el árbol, ya que ira recorriendo solamente por la izquierda del árbol.

■ eliminarPalabra

Para este método hacemos uso del método anterior, *removeMin*.

- Algoritmo **ABBPalabras**:

```
/**
 * Elimina del rbol la palabra pasada como par metro
 * Pre: la palabra como mucho esta una vez en el
 *diccionario
 * @param pal: palabra a eliminar
 */

private void eliminarPalabra(Palabra pal) {
    if (!this.isEmpty()) {
        this.root = this.root.eliminarPalabra(pal);
    }
}
```

Listing 2.17: Elimina del árbol la palabra pasada como parámetro.

- Algoritmo **NodoABBPalabras**:

```
public NodoABBPalabras eliminarPalabra(Palabra pal) {
    int comp = pal.compareTo(this.info);
    if (comp == 0) {          // Caso (a): this es el nodo a
        eliminar
        if (!this.hasLeft())
            return this.right;      // Caso (a1)
        else if (!this.hasRight())
            return this.left;       // Caso (a2)
        else {                   // Caso (a3): Tiene los dos
            subarboles, sustituir por el valor minimo del
            // subarbol derecho
            ResultadoRemoveMin min = this.right.removeMin();
            this.right = min.elNodo;
            return this;
        }
    } else if (comp < 0) {      // Caso (b) El elemento a
        eliminar, si esta, estar en el subarbol izq
        if (this.hasLeft())
            this.left = this.left.eliminarPalabra(pal);
        return this;
    } else {                   // comp>0: Caso (c) El elemento a
        eliminar, si esta, estara en el subarbol dcho
        if (this.hasRight())
            this.right = this.right.eliminarPalabra(pal);
        return this;
    }
}
```

Listing 2.18: Elimina el mínimo del arbol.

Tres opciones:

a) **Son iguales:** el nodo a eliminar es el actual (*this*). Otros tres casos:

a1) *this* no tiene subárbol izquierdo:

-Su padre debe apuntar al subárbol derecho.

a2) *this* tiene subárbol izquierdo pero no derecho:

-Su padre debe apunrar al subárbol izquierdo.

a3) *this* tiene subárbol izquierdo y derecho:

-Eliminar el nodo menor del subárbol derecho de *this* y poner como valor de *this* el que tenía el nodo eliminado.

-Su padre debe seguir apuntando a *this*.

b) Elem es menor que *this.info*, estará en subárbol izquierdo:

-Si tiene subárbol izquierdo, entonces llamará recursivamente al método desde *this.left* y actualizará *this.left*.

-Devolverá *this*.

c) Elem es mayor que *this.info*, estará es subárbol derecho: -Si tiene subárbol derecho, entonces llamará recursivamente al método desde *this.right* y actualizará *this.right*. -Devolverá *this*.

- Casos de prueba considerados:

Entrada	Función
adult	Si la palabra adult entra como parámetro, compara con la palabra del root para saber si tiene que hacer la búsqueda en la rama de la derecha o en la rama de la izquierda y cuando encuentra esa plabra la elimina.
dormir	Como la palabara pasada como parámetro no está en el árbol, no hace nada.

- Orden temporal:

El orden temporal es de $O(\log n)$ (cuando perfectamente equilibrado) y en el caso peor es de $O(n)$.

- **filtrarPalabrasClave**

- Algoritmo:

```
/**
 * Haciendo uso de los m todos anteriores, obtiene la
 * lista de palabras del rbol a eliminar y elimina cada
 * una de ellas.
 *
 */

public void filtrarPalabrasClave() {
    LinkedList<Palabra> lista = obtenerPalabrasAEliminar();

    for(Palabra i: lista) {
        eliminarPalabra(i);
    }
}
```

Listing 2.19: obtiene la lista de las palabras a eliminar del árbol y las elimina.

- Casos de prueba considerados:

Entrada	Función
dormir	Si está la palabra en la lista de las palabras a eliminar, haciendo uso de los métodos anteriores elimina dicha palabra de la lista.
adult	Como adult es una palabra clave, no va a estar en la lista de palabrasAEliminar, por tanto, devuelve null.

- Orden temporal:

El orden temporal es $O(N)$ siendo N el número de elementos en el árbol.

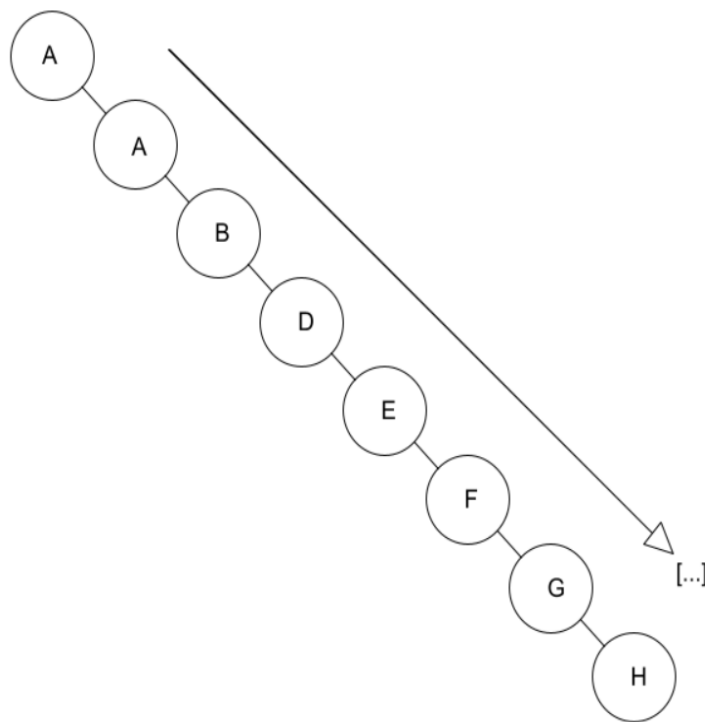
Capítulo 3

Respuesta al apartado f.

El apartado f era el siguiente:

Responder a la siguiente pregunta: *¿Qué habría pasado si hubiéramos trabajado con el fichero words.txt original, en el que las palabras estaban ordenadas alfabéticamente?*

Si el fichero de entrada estuviese ordenado, nuestro árbol quedaría de esta manera:



Estaría ordenado en función lexicográfica de manera que la raíz es la primera palabra, y los posteriores a esta, estarían en la derecha del árbol, dejando el lado izquierdo del árbol libre. Siendo una recta como se visualiza en la imagen. Teniendo

